

Greedy Heuristics for the Traveling Salesman Problem: a Short Tutorial on the TSPLIB Benchmarks

Heuristic Algorithms in Transport and Finance — Master’s course assignment

Javier Salvatierra
Jesús Inurria

September 2026

Abstract

The Traveling Salesman Problem (TSP) asks for the shortest closed tour through n cities. We study two constructive greedy heuristics, *nearest neighbour* and *greedy edge* (multi-fragment), on the 111 TSPLIB instances of the assignment, from 14 to 85 900 cities. Both run on a candidate graph with the ten nearest neighbours of each city, so no $n \times n$ distance matrix is built. On the 31 instances with a known optimal tour, greedy edge is on average 18.7% above the optimum (nearest neighbour: 26.5%); it gives the shorter tour on 93 of the 111 instances and builds the 85 900-city tour in about 0.41 s. Its weak points are the final joining of fragments and instances with many tied distances, illustrated by the pathological instance **brg180**.

1 Introduction

A salesman must visit n cities exactly once and return to the start at minimum total distance. The TSP has applications in logistics, robotics, DNA sequencing and urban planning [2]. Exact methods scale poorly: the largest TSPLIB instance solved to optimality, with 85 900 cities, needed about 569 hours of wall-clock time, more than 136 CPU-years [2]. Large instances are therefore attacked with heuristics; the machine-learning constructive heuristic reviewed in [2], for instance, is built on the Clarke–Wright savings rule and compared with multi-fragment and furthest insertion. Here we implement the two simplest constructive heuristics described by Jünger et al. [1] and evaluate them on the whole TSPLIB collection [3] of the assignment: 111 symmetric instances, 32 of which come with a known optimal tour, our reference solution. The rest of the paper (i) states the model, (ii) explains the choice of heuristics and (iii) reports times, quality and tours.

2 The TSP model

2.1 Parameters and decision variables

Let $V = \{1, \dots, n\}$ be the cities and E the set of edges $e = \{i, j\}$ between pairs of cities, with integer length $d_e = d_{ij} = d_{ji}$. The decision variable $x_e \in \{0, 1\}$ equals 1 when edge e belongs to the tour. For $S \subset V$, $E(S)$ denotes the edges with both ends in S and $\delta(i)$ the edges incident to city i .

2.2 Formulation

$$\min_x \sum_{e \in E} d_e x_e \quad \text{s.t.} \quad \sum_{e \in \delta(i)} x_e = 2 \quad \forall i \in V, \quad \sum_{e \in E(S)} x_e \leq |S| - 1 \quad \forall S \subset V, \quad 2 \leq |S| \leq n - 1, \quad x_e \in \{0, 1\}. \quad (1)$$

Read literally: the objective is the total length of the chosen edges; the degree equations say that every city is joined to exactly two neighbours in the tour; the subtour-elimination inequalities forbid closed loops through a subset S of the cities [1, 2]. They contain the two rules of every constructive heuristic below: an edge may be added only while both of its ends have degree less than 2, and only if it closes no loop before all cities are connected. The quality of a tour of length L is its *gap* $100(L - L_{\text{opt}})/L_{\text{opt}}$ to the optimum.

2.3 Complexity and benchmark instances

The TSP is NP-hard [2]: there are $(n - 1)!/2$ different tours, and the Held–Karp dynamic programme needs $O(n^2 2^n)$ time and $O(n 2^n)$ memory [2]. A constructive heuristic instead builds a single tour with a simple local rule and no backtracking. It offers no optimality guarantee: nearest neighbour can be a factor $O(\log n)$ longer than the optimum on adversarial instances [1].

TSPLIB [3] fixes each integer distance d_{ij} through **EDGE_WEIGHT_TYPE**: rounded Euclidean (**EUC_2D**, 78 instances), Euclidean rounded up (**CEIL_2D**, 4), pseudo-Euclidean (**ATT**, 2), geographical distance on the sphere (**GEO**, 10) or an explicit matrix (**EXPLICIT**, 17). Only these exact formulas reproduce the published optima (Section 4). One instance, **linhp318**, has a fixed edge that every tour must contain.

3 Choice of heuristic

Table 1 lists the classical constructive heuristics of [1]. We chose the two simplest, both *greedy*: at each step they take the locally cheapest feasible decision and never undo it. Being purely constructive, their tours are longer than those of savings or insertion; Section 5 quantifies by how much.

The decisive argument is the size range of the task: 111 instances of up to 85 900 cities.

- **Candidate graph.** Storing all $n(n - 1)/2$ distances is impossible for 85 900 cities ($\approx 3.7 \cdot 10^9$ pairs). Following [1], both heuristics use the $k = 10$ nearest neighbours of each city, found with a k-d tree, so time and memory stay close to linear.

Table 1: Constructive heuristics considered. Gaps in [1] are averages over 30 Euclidean TSPLIB problems of 105–2392 cities; “Gap here” is measured in this work (Section 5).

Heuristic	Greedy rule	Time	Gap in [1]	Gap here	Comment
Nearest neighbour	closest unvisited city	$O(n^2)$	24.2 % (18.6 % with candidates)	26.5 %	Chosen ; baseline
Greedy edge	shortest edge keeping degrees ≤ 2 and no loop	$O(kn \log kn)$	not reported	18.7 %	Chosen ; global view
Farthest insertion	insert the farthest city at its best position	$O(n^2)$	9.9 %	—	better, but quadratic
Savings (Clarke–Wright)	merge the two paths with the largest saving	$O(n^2 \log n)$	9.8 % (9.6 % with candidates)	—	best; harder to code

Algorithm 1 Greedy edge (multi-fragment)

Input: cities V , candidate edges C (k nearest neighbours of every city), lengths d

- 1: $\deg(i) \leftarrow 0$ for all i ; every city is a fragment of its own (union–find)
- 2: $F \leftarrow$ edges of C sorted by increasing d_e (all edges if $n \leq 2500$)
- 3: **repeat**
- 4: **for** each edge $e = \{i, j\}$ of F , in order **do**
- 5: **if** $\deg(i) < 2$ **and** $\deg(j) < 2$ **and** i, j lie in different fragments **then**
- 6: add e to the tour; increase $\deg(i), \deg(j)$; merge the two fragments
- 7: $F \leftarrow$ edges between the 8 nearest *endpoints* of each fragment endpoint, sorted by length
- 8: **until** $n - 1$ edges have been added
- 9: close the resulting path into a cycle

- **Two meanings of “greedy”.** Nearest neighbour is greedy over *cities*: it grows one path, always moving to the closest unvisited city. Greedy edge is greedy over *edges*: it sorts them by length and grows many fragments at once.

4 Implementation

The code is a Python notebook (`tsp_greedy_en.ipynb`, delivered separately) using Python 3.12.3, NumPy 2.4.4, SciPy (k-d tree), pandas and matplotlib. It reads the TSPLIB files (five distance functions, fixed edges), builds the candidate graph with $k = 10$ and runs both heuristics. *Nearest neighbour* starts at city 1 and moves to the first unvisited city in the sorted candidate list of the current city, scanning all free cities only if every candidate is visited, so it returns exactly the classical tour. *Greedy edge* (Algorithm 1) uses a degree counter and a union–find structure to enforce the two rules of Section 2.2: candidate edges are processed by increasing length (phase 1) and the remaining fragments are joined by repeating the rule on their endpoints only, with the 8 nearest endpoints of each and a rebuilt k-d tree per round (phase 2). For $n \leq 2500$ all edges replace the candidates, which gives the exact greedy edge.

Checks. (i) The length computed for each of the 32 optimal tours matches the value declared in its file in 25 of the 26 cases that declare one; the exception, `tsp225`, differs by 3 units (3916 against 3919), apparently an outdated comment. (ii) On the 81 instances of up to 1100 cities without fixed edges, greedy edge with all edges coincides with a naive $O(n^2 \log n)$ implementation in 81 of 81 cases; all tours are validated as permutations of the cities (fixed edges included). Runs are single-threaded on a single-vCPU Intel Xeon (2.1 GHz) virtual machine (Linux); times are medians of three runs and exclude the candidate graph (at most 0.18 s), except that exact greedy edge builds its own neighbour lists. Sanity check: on `eil51` (optimum 426) nearest neighbour gives 511 (+20.0 %) in 0.04 ms and greedy edge 531 (+24.6 %) in 0.78 ms (Figure 1a–b).

5 Computational results

Table 2 collects the results by instance size and Figure 1 shows four tours (a–d) and the statistics over all instances (e–g).

Accuracy. On the 31 instances with known optimum, greedy edge has a mean gap of 18.7 % (median 18.7 %) against 26.5 % (median 24.5 %) for nearest neighbour. It gives the shorter tour on 93 of the 111 instances (mean length ratio 0.962, median 0.938), and the advantage is largest on the biggest instances: for $n > 5000$ it wins in all 10 cases with a ratio of 0.919, whereas for 101–500 cities the two are practically tied (Table 2). Figure 1a–d shows why: both build short local connections, but nearest neighbour leaves cities behind that must be reached later with long edges, whereas greedy edge produces a few very long edges where the fragments are finally joined.

Computational time. Building a tour is fast: no single tour takes more than 4.9 s (greedy edge on `pr2392`, in exact mode). For `pla85900` greedy edge needs 0.41 s and nearest neighbour 1.9 s, plus 0.18 s for the candidate graph. Nearest neighbour is close to linear up to 2500 cities (log–log slope 1.0) but its slope rises to 1.8 above, which we attribute to the exhaustive scan it performs when all candidates are visited. Greedy edge grows steeply while it uses all edges (slope 2.5 up to $n = 2500$) and almost linearly with candidates (slope 1.1), so it is the faster of the two from about 12 000 cities on (Table 2, Figure 1g).

Where greedy edge fails. It loses to nearest neighbour on 18 instances, on 5 of them by more than 5 %. The extreme case is `brg180`: only six distinct distances (0, 20, 30, 3500, 9000, 10 000) and thousands of ties. Greedy edge takes the cheap edges first (all 90 of length 0 and 75 of length 20), corners itself and pays four edges of length 9000: 37 830 against an optimum of 1950 (nearest neighbour: 12 360); with it, the mean gaps would be 75.6 % and

Table 2: Nearest neighbour (NN) against greedy edge (GE) by instance size. Gaps: averages over the instances with known optimum (**brg180** excluded); ratio: $\text{length}(\text{GE})/\text{length}(\text{NN})$ averaged over the instances of the class; times: means of the per-instance medians, in ms.

Cities n	Instances	With opt.	NN gap (%)	GE gap (%)	Ratio	GE wins	NN time	GE time
≤ 100	28	18	26.5	19.0	0.948	20	0.06	1.15
101–500	38	9	26.8	17.9	1.006	30	0.27	19.2
501–1000	12	2	27.2	17.5	0.924	12	1.06	178
1001–5000	23	2	24.5	19.6	0.947	21	2.80	1545
> 5000	10	0	–	–	0.919	10	260	95.7
All	111	31	26.5	18.7	0.962	93	24.3	355

42.4 %. The candidate graph is also an approximation: with $k = 10$ the tour differs from the exact greedy edge in 42 of the 81 small instances (up to $\pm 10\%$ on single instances, -0.1% on average), hence the exact version up to 2500 cities.

6 Conclusions

Nearest neighbour and greedy edge build a tour for every instance of the collection in at most 4.9 s, even for 85 900 cities, provided they work on a candidate graph instead of the full distance matrix. Greedy edge is clearly the better one (average gap 18.7 % against 26.5 %), and its global view pays off on large instances. The weak spots are the final joining of fragments and instances with many ties. Natural next steps are a local-improvement phase (2-opt, Or-opt), joining the fragments with the savings rule, the best constructive heuristic in [1], and learned guidance of the candidate edges as in the hybrid methods reviewed in [2].

Code availability. All computations are reproducible from the notebook `tsp_greedy_en.ipynb`, delivered separately: unzip `ALL_tsp` next to it and *Run All* (≈ 1 min).

References

- [1] M. Jünger, G. Reinelt, G. Rinaldi. The traveling salesman problem. In *Handbooks in Operations Research and Management Science*, 7:225–330, 1995.
- [2] E. Alanzi, M. E. B. Menai. Solving the traveling salesman problem with machine learning: a review of recent advances and challenges. *Artif. Intell. Rev.*, 58(9):267, 2025.
- [3] G. Reinelt. TSPLIB — a traveling salesman problem library. *ORSA J. Comput.*, 3(4):376–384, 1991. Data: comopt.ifl.uni-heidelberg.de/software/TSPLIB95

figures/fig_main.pdf

Figure 1: Tours (a–d) and statistics (e–g). (a–b) **eil51** ($n = 51$, optimum 426) and (c–d) **pcb442** ($n = 442$, optimum 50 778): nearest neighbour (orange) and greedy edge (blue); titles give the tour length and its gap to the optimum. (e) Gap on the 31 instances with known optimum (**brg180** excluded). (f) Length of the greedy-edge tour over that of the nearest-neighbour tour, all 111 instances (below 1: greedy edge wins). (g) Time per heuristic (median of three runs, candidate graph excluded); the dashed line marks where greedy edge switches from all edges to candidate edges.